Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Conmutación y Enrutamiento de redes

Maestro:

Zuriel Dathan Mora Felix

Tema:

Procesamiento de archivo TXT

Nombre de los alumnos:

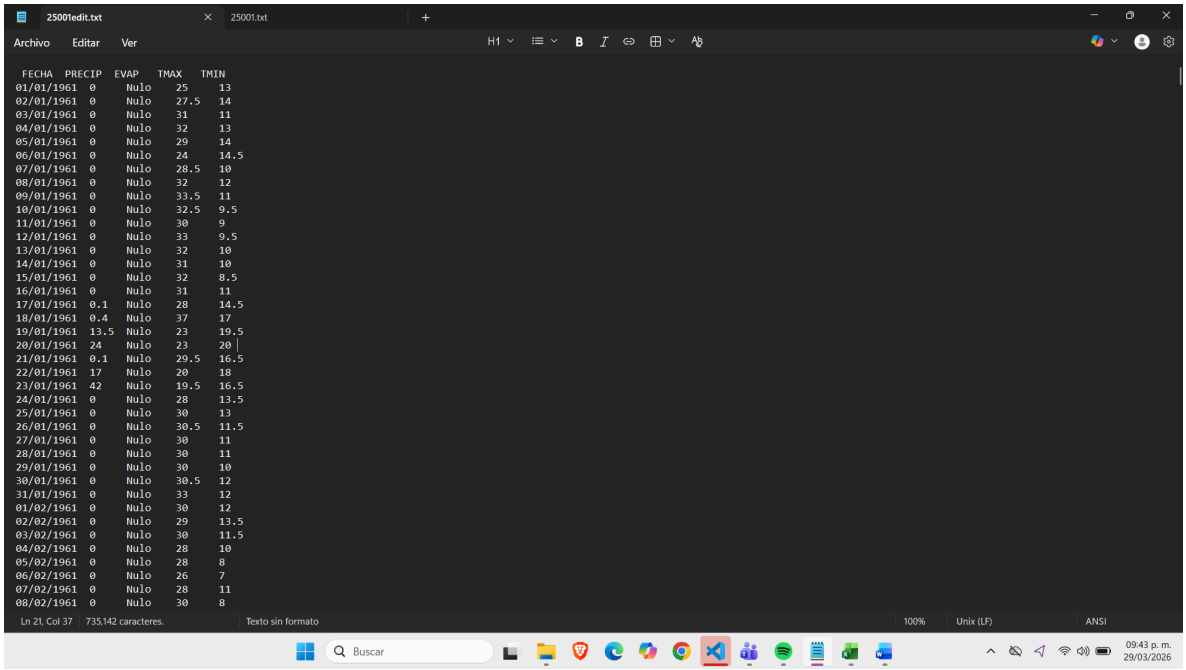
Bernardo Daniel Bejarano Felix

Ricardo de la Mora Morales

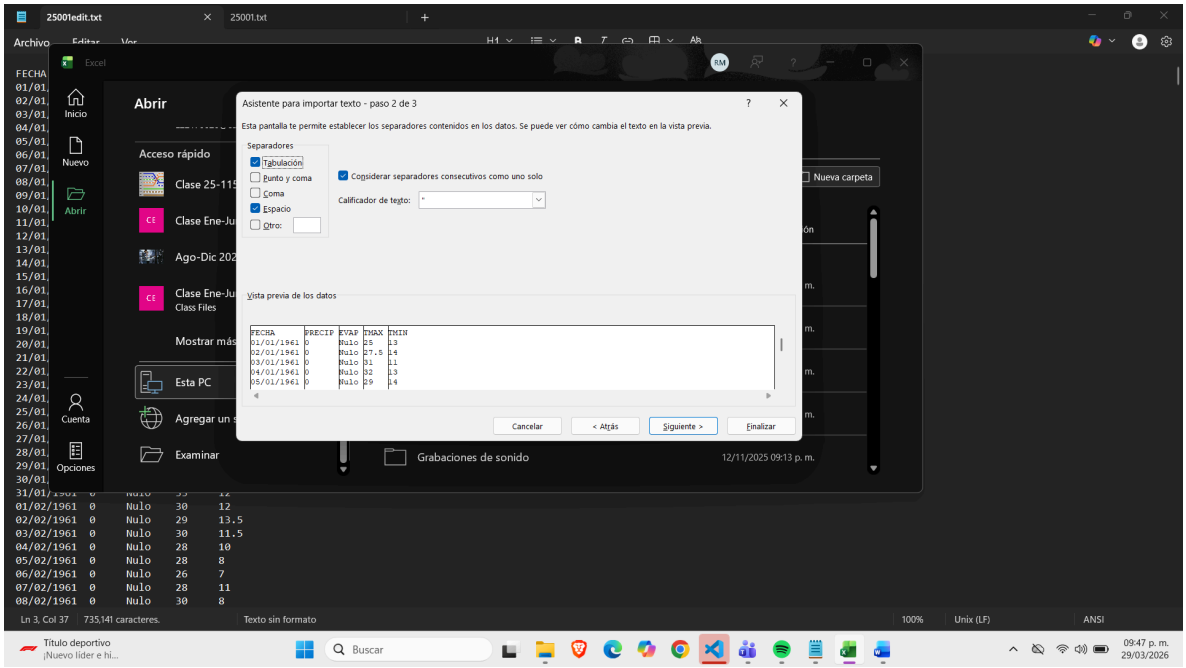
Fecha de entrega:

31/03/2026

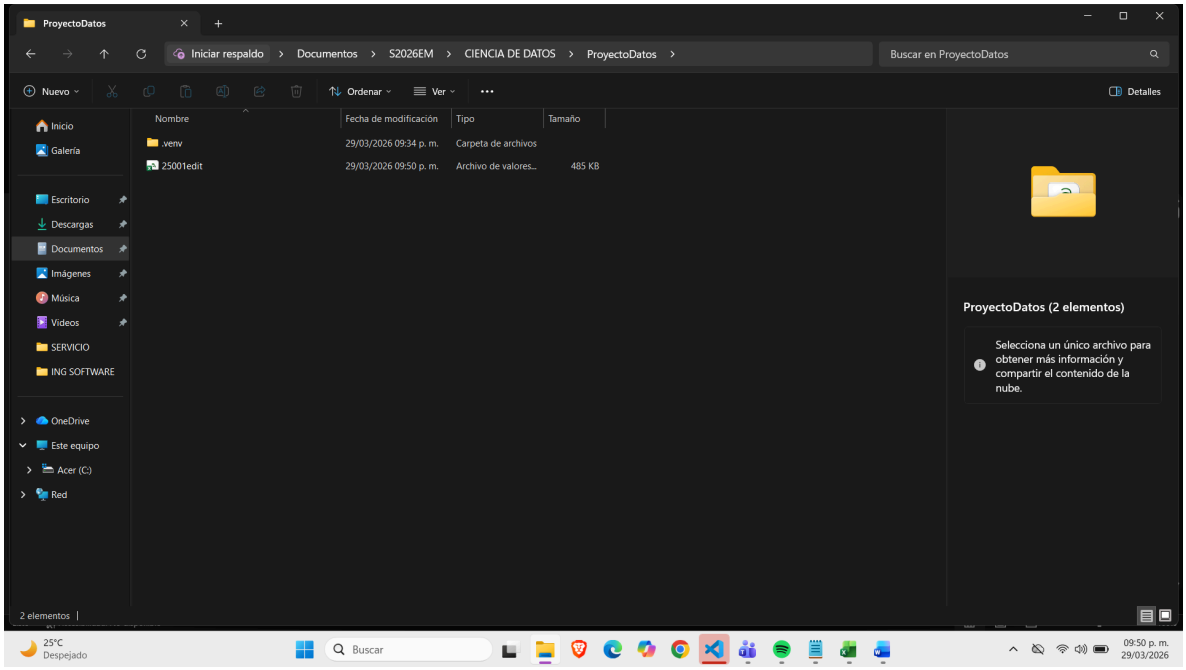
Descargamos el archivo .txt y lo editamos para quitarle texto que viene de más y sólo dejarle lo que necesitamos para este proyecto. Desde aquí vemos nuestras columnas que son FECHA, PRECIPITACIÓN, EVAPORACIÓN, TEMPERATURA MAXIMA, TEMPERATURA MINIMA.



Lo abrimos en Excel para convertir el archivo en .csv, para eso vamos a configurar que es delimitado por tabulación y nos dé el formato de tabla que se ocupa.



Al abrir nos da el formato de tabla que queremos listo para descargar en archivo .csv y guardarlo en la carpeta que usaremos para trabajar.



Lo abrimos en Visual Studio Code ya con el archivo .csv y creamos una celda donde importamos pandas y cargamos el archivo en una variable para manejar los datos a partir de pandas. Para empezar mostramos los primeros 5 registros y los últimos 5 registros para ver nuestro dataframe.

```python
# Cargamos Pandas para abir el archivo CSV y ponemos la primera columna como indice ya que es una serie de tiempo
import pandas as pd
df = pd.read_csv("25001edit.csv", index_col=0)
# Mostramos los primeros 5 registros
df.head(5)
```

[70] ✓ 0.1s

FECHA	PRECIP	EVAP	TMAX	TMIN
01/01/1961	0	Nulo	25	13
02/01/1961	0	Nulo	27.5	14
03/01/1961	0	Nulo	31	11
04/01/1961	0	Nulo	32	13
05/01/1961	0	Nulo	29	14

```python
# Mostramos los ultimos 5 registros
df.tail(5)
```

[71] ✓ 0.0s

FECHA	PRECIP	EVAP	TMAX	TMIN
27/07/2018	2.3	0.6	37	19
28/07/2018	9.8	3.7	37	23.5
29/07/2018	6	3.3	35	20
30/07/2018	48	6	36	22
31/07/2018	2	3.7	34	22

Realizamos un perfilado de datos breve, sin embargo, vemos algo de suma importancia y es que el tipo de datos de la columna vienen en formato str y no está encontrando ningún nulo cosa que no es correcta ya que cuando ejecutamos las primeras 5 celdas nos enseña nulos. Debemos de hacer una transformación para adecuar los tipos de datos correspondientes. Esto lo puede tomar así ya que en las columnas hay campos con texto “Nulo”, lo que ocasiona que todas las columnas los tome como STR y esto se valida utilizando los métodos como .describe, .isnull().sum, .shape.

```python
# Mostramos informacion general como el tipo de datos, valores nulos, etc
df.info()
```

[46] ✓ 0.0s Python

```
<class 'pandas.DataFrame'>
Index: 19575 entries, 01/01/1961 to ------------------------------------
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   PRECIP  19574 non-null  str
 1   EVAP    19574 non-null  str
 2   TMAX    19574 non-null  str
 3   TMIN    19574 non-null  str
dtypes: str(4)
memory usage: 764.6+ KB
```

```python
# Mostramos el numero de filas y columnas
print(df.shape)
```

[44] ✓ 0.0s Python

```
(19575, 4)
```

```python
# Mostramos el numero de nulos
df.isnull().sum()
```

[47] ✓ 0.0s Python

```
PRECIP    1
EVAP      1
TMAX      1
TMIN      1
dtype: int64
```

```python
# Mostramos las descripciones estadisticas de las columnas numericas
df.describe()
```

[48] ✓ 0.0s Python

	PRECIP	EVAP	TMAX	TMIN
count	19574	19574	19574	19574
unique	493	135	60	63
top	0	3	35	23
freq	16087	401	1980	1225

Remplazamos “Nulo” con valores nulos que es lo que nos servirá para después hacer las conversiones de los datos y no lo tome como STR. Aquí utilizamos Numpy que es una librería que ya hemos usado en el curso. Es bastante útil para el procesamiento de datos ya que facilita la creación, selección, ordenación y transformación de grandes volúmenes de datos numéricos. Utilizamos la función de Python que nos sirve para reemplazamiento que es .replace() y donde utilizamos la función de Numpy: np.nan. Finalmente, reemplazamos los valores “Nulo” por np.nan y agregando inplace=True para que se guarde en nuestro dataframe.

```python
# Cargamos Numpy ya que cuenta con funciones que nos va a servir para nuestro preprocesamiento de datos
import numpy as np
# Reemplazamos los valores "Nulo" por NaN para que Pandas los reconozca como valores faltantes
df.replace("Nulo", np.nan, inplace=True)
# Volvemos a mostrar la informacion general para verificar que los valores "Nulo" han sido reemplazados por NaN
df.info()
```

[ ] Python

En este punto ya nos aparecen los datos tipo nulo y lo podemos corroborar con la función .isnull().sum()

```python
# Mostramos el numero de nulos (ahora ya existen)
df.isnull().sum()
```

✓ 0.0s

```
PRECIP     40
EVAP      272
TMAX        5
TMIN        6
dtype: int64
```

Convertimos nuestras columnas a datos numéricos a “Float64” ya que es lo adecuado una vez revisando nuestros campos. Utilizamos pd.to_numeric() que es una función de Pandas que convierte los tipos de datos a int o float.

```python
# Convertimos las columnas a tipo numerico para poder trabajar con ellas
df["PRECIP"] = pd.to_numeric(df["PRECIP"])
df["EVAP"] = pd.to_numeric(df["EVAP"])
df["TMIN"] = pd.to_numeric(df["TMIN"])
df["TMAX"] = pd.to_numeric(df["TMAX"])
```

[64] ✓ 0.0s Python

Una vez procesados nuestros datos vamos a realizar un perfilado de datos completo.

Mostramos los primeros 5 registros y los últimos 5 registros de nuestro dataframe.

```python
# Mostramos los primeros 5 registros una vez procesados los datos
df.head(5)
```

✓ 0.0s

	PRECIP	EVAP	TMAX	TMIN
FECHA				
01/01/1961	0.0	NaN	25.0	13.0
02/01/1961	0.0	NaN	27.5	14.0
03/01/1961	0.0	NaN	31.0	11.0
04/01/1961	0.0	NaN	32.0	13.0
05/01/1961	0.0	NaN	29.0	14.0

```python
# Mostramos las ultimas 5 registros una vez procesados los datos
df.tail(5)
```

✓ 0.0s

	PRECIP	EVAP	TMAX	TMIN
FECHA				
27/07/2018	2.3	0.6	37.0	19.0
28/07/2018	9.8	3.7	37.0	23.5
29/07/2018	6.0	3.3	35.0	20.0
30/07/2018	48.0	6.0	36.0	22.0
31/07/2018	2.0	3.7	34.0	22.0

Mostramos un resumen técnico incluyendo el número de filas, columnas, tipos de datos (dtypes) y el recuento de valores no nulos.

```python
# Mostramos un resumen de los datos una vez procesados los datos
df.info()
```

[69] ✓ 0.0s Python

```
<class 'pandas.DataFrame'>
Index: 19575 entries, 01/01/1961 to ------------------------------------
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   PRECIP  19534 non-null  Float64
 1   EVAP    19302 non-null  Float64
 2   TMAX    19569 non-null  Float64
 3   TMIN    19568 non-null  Float64
dtypes: Float64(4)
memory usage: 841.1+ KB
```

```python
# Mostramos el numero de filas y columnas
df.shape
```

[70] ✓ 0.0s Python

```
(19575, 4)
```

Mostramos un resumen de estadísticas clave como la media, desviación estándar, valores mínimos/máximos y percentiles para columnas numéricas.

```python
# Mostramos las descripciones estadisticas una vez procesados los datos
df.describe()
```

[74] ✓ 0.0s Python

	PRECIP	EVAP	TMAX	TMIN
count	19534.0	19302.0	19569.0	19568.0
mean	2.217452	4.882157	34.468808	16.65742
std	9.015782	2.27934	3.430054	6.248605
min	0.0	0.0	18.0	-2.5
25%	0.0	3.2	32.5	11.0
50%	0.0	4.5	35.0	16.5
75%	0.0	6.5	37.0	23.0
max	217.4	15.6	45.5	29.0

Una vez realizado este perfilado de datos observamos que nuestra transformación de datos fue un éxito ya que se logró realizar el cambio de la palabra “Nulo” por valores nulos reales, donde ya se muestra una completa perfilado de datos y es de mucha ayuda al momento de realizar un análisis de datos. Asimismo, cambiar el tipo de dato de las columnas de STR a su tipo de dato numérico nos da una mejor representación al momento de hacer un perfilado de datos.

En el siguiente bloque de código completamos los solicitado en el punto 2 de la tarea **“Contar total de registros faltantes por fecha”,** recorremos cada una de las fechas con un ciclo for y a través de una resta del total de datos por fila menos los elementos válidos (no nulos) calculamos los elementos nulos por fecha.

```python
# "2.- Contar total de registros faltantes por fecha"

# Obtenemos todas las filas que vamos a trabajar
total_filas = df.shape[0]

# Por medio de un for interamos sobre cada fila para contar el numero de nulos que tienen por fila
for i in range(total_filas):
    fila = df.iloc[i]

    total_columnas = fila.shape[0]                    # Ya sabemos que es 4 pero lo correcto es calcularlo
    elementos_validos = fila.dropna().shape[0]        # Contamos solo los elementos no nulos
    faltantes = total_columnas - elementos_validos    # Calculamos el numero de nulos

    # Mostramos solo si hay nulos para no saturar la pantalla
    if faltantes > 0:
        print(f"{df.index[i]} | Registros faltantes: {faltantes}")
```

[112] ✓ 5.6s

```
01/01/1961 | Registros faltantes: 1
02/01/1961 | Registros faltantes: 1
03/01/1961 | Registros faltantes: 1
04/01/1961 | Registros faltantes: 1
05/01/1961 | Registros faltantes: 1
06/01/1961 | Registros faltantes: 1
07/01/1961 | Registros faltantes: 1
08/01/1961 | Registros faltantes: 1
09/01/1961 | Registros faltantes: 1
10/01/1961 | Registros faltantes: 1
11/01/1961 | Registros faltantes: 1
12/01/1961 | Registros faltantes: 1
13/01/1961 | Registros faltantes: 1
14/01/1961 | Registros faltantes: 1
15/01/1961 | Registros faltantes: 1
16/01/1961 | Registros faltantes: 1
17/01/1961 | Registros faltantes: 1
18/01/1961 | Registros faltantes: 1
```

Ahora realizamos una imputación de datos donde vamos a rellenar los valores nulos por la media de cada columna, con esto cumplimos el punto 1 de la tarea **“Eliminar rellenar nulos con la media de la columna”**.

```python
# "1.- Eliminar rellenar nulos con la media de la columna"

# Imputamos los valores faltantes con la media de cada columna como se solicita en la asigancion
df["PRECIP"] = df['PRECIP'].fillna(df['PRECIP'].mean())
df["EVAP"] = df['EVAP'].fillna(df['EVAP'].mean())
df["TMAX"] = df['TMAX'].fillna(df['TMAX'].mean())
df["TMIN"] = df['TMIN'].fillna(df['TMIN'].mean())
```

✓ 0.0s

Realizamos un perfilado de datos breve donde se muestra nuestra imputación.

Mostramos el resumen estadístico donde nos muestra la media de cada columna.

```python
# Mostramos un resumen estadistico una vez imputados los valores faltantes
df.describe()
```

[81] ✓ 0.0s Python

	PRECIP	EVAP	TMAX	TMIN
count	19575.0	19575.0	19575.0	19575.0
mean	2.217452	4.882157	34.468808	16.65742
std	9.006335	2.263389	3.429528	6.247488
min	0.0	0.0	18.0	-2.5
25%	0.0	3.2	32.5	11.0
50%	0.0	4.6	35.0	16.5
75%	0.0	6.4	37.0	23.0
max	217.4	15.6	45.5	29.0

Mostramos los nulos. Se demuestra que no quedó ningún nulo después de la imputación.

```python
# Mostramos el numero de nulos una vez imputados los valores faltantes
df.isnull().sum()
```

[82] ✓ 0.0s Python

```
PRECIP    0
EVAP      0
TMAX      0
TMIN      0
dtype: int64
```

Final

Con esto habremos perfilado y limpiado nuestros datos para empezar a trabajarlos según las necesidades del cliente/problema con el que estemos trabajando.